

06 — Query SQL (CRUD, JOIN, Agregasi)

Tujuan: Mendemonstrasikan berbagai jenis query SQL yang relevan dengan kebutuhan bisnis BelajarKUY.

Kategori Query

1. [CRUD Dasar](#)
2. [JOIN antar tabel](#)
3. [Agregasi & Group By](#)
4. [Subquery](#)
5. [Query untuk Dashboard](#)
6. [Query Lanjutan \(Window, CTE\)](#)

1. CRUD Dasar

1.1 CREATE — Tambah Data

```
-- Tambah kategori baru
INSERT INTO categories (name, slug, description, status, created_at, updated_at)
VALUES ('Pemrograman', 'pemrograman', 'Kursus seputar coding', 1, NOW(), NOW());

-- Tambah user (instruktur)
INSERT INTO users (name, email, password, role, created_at, updated_at)
VALUES ('Budi Programmer', 'budi@example.com',
        '$2y$10$hashedpassword...', 'instructor', NOW(), NOW());

-- Tambah kursus
INSERT INTO courses (category_id, instructor_id, title, slug, price, status,
created_at, updated_at)
VALUES (1, 2, 'Belajar Laravel dari Nol', 'belajar-laravel-dari-nol',
        199000, 'active', NOW(), NOW());
```

1.2 READ — Ambil Data

```
-- Ambil semua kursus aktif
SELECT id, title, price, thumbnail
FROM courses
WHERE status = 'active'
ORDER BY created_at DESC;

-- Ambil 1 kursus dengan slug tertentu
SELECT *
FROM courses
WHERE slug = 'belajar-laravel-dari-nol'
```

```
LIMIT 1;

-- Ambil instruktur saja
SELECT id, name, email, photo
FROM users
WHERE role = 'instructor';
```

1.3 UPDATE — Ubah Data

```
-- Ubah harga kursus
UPDATE courses
SET price = 149000, updated_at = NOW()
WHERE id = 1;

-- Aktifkan kupon
UPDATE coupons
SET status = 1, updated_at = NOW()
WHERE code = 'NEWUSER10';

-- Tandai kursus sebagai bestseller
UPDATE courses
SET bestseller = 1
WHERE id IN (1, 5, 12);
```

1.4 DELETE — Hapus Data

```
-- Hapus kursus tertentu (akan cascade ke sections, lectures, dst)
DELETE FROM courses WHERE id = 99;

-- Hapus item dari wishlist user
DELETE FROM wishlists
WHERE user_id = 3 AND course_id = 7;
```

2. JOIN Antar Tabel

2.1 INNER JOIN — Kursus dengan Nama Kategori & Instruktur

```
SELECT
  c.id,
  c.title,
  c.price,
  cat.name      AS kategori,
  sub.name      AS sub_kategori,
  u.name        AS instruktur
FROM courses c
```

```
INNER JOIN categories cat ON c.category_id = cat.id
LEFT JOIN sub_categories sub ON c.subcategory_id = sub.id
INNER JOIN users u ON c.instructor_id = u.id
WHERE c.status = 'active'
ORDER BY c.created_at DESC;
```

2.2 LEFT JOIN — Kursus & Rata-rata Rating

```
SELECT
    c.id,
    c.title,
    COALESCE(ROUND(AVG(r.rating), 2), 0) AS avg_rating,
    COUNT(r.id) AS total_review
FROM courses c
LEFT JOIN reviews r
    ON r.course_id = c.id
    AND r.status = 'approved'
WHERE c.status = 'active'
GROUP BY c.id, c.title
ORDER BY avg_rating DESC;
```

2.3 Multi-JOIN — Detail Order

```
SELECT
    o.id AS order_id,
    p.midtrans_order_id,
    p.status AS payment_status,
    u.name AS pembeli,
    c.title AS kursus,
    inst.name AS instruktur,
    cp.code AS kupon,
    o.original_price,
    o.discount_amount,
    o.final_price,
    o.created_at
FROM orders o
JOIN payments p ON o.payment_id = p.id
JOIN users u ON o.user_id = u.id
JOIN courses c ON o.course_id = c.id
JOIN users inst ON o.instructor_id = inst.id
LEFT JOIN coupons cp ON o.coupon_id = cp.id
ORDER BY o.created_at DESC;
```

3. Agregasi & GROUP BY

3.1 Total Pendapatan per Instruktur

```

SELECT
  u.id,
  u.name                                AS instruktur,
  COUNT(o.id)                          AS total_transaksi,
  SUM(o.final_price)                   AS total_pendapatan,
  AVG(o.final_price)                   AS rata_rata
FROM users u
JOIN orders o ON o.instructor_id = u.id
WHERE o.status = 'completed'
GROUP BY u.id, u.name
ORDER BY total_pendapatan DESC;

```

3.2 Kursus Terlaris (Top 10)

```

SELECT
  c.id,
  c.title,
  COUNT(o.id)                        AS total_terjual,
  SUM(o.final_price)                 AS revenue
FROM courses c
JOIN orders o
  ON o.course_id = c.id
  AND o.status = 'completed'
GROUP BY c.id, c.title
ORDER BY total_terjual DESC
LIMIT 10;

```

3.3 Distribusi Rating per Kursus

```

SELECT
  c.title,
  SUM(CASE WHEN r.rating = 5 THEN 1 ELSE 0 END) AS bintang_5,
  SUM(CASE WHEN r.rating = 4 THEN 1 ELSE 0 END) AS bintang_4,
  SUM(CASE WHEN r.rating = 3 THEN 1 ELSE 0 END) AS bintang_3,
  SUM(CASE WHEN r.rating = 2 THEN 1 ELSE 0 END) AS bintang_2,
  SUM(CASE WHEN r.rating = 1 THEN 1 ELSE 0 END) AS bintang_1,
  COUNT(r.id)                                AS total_review,
  ROUND(AVG(r.rating), 2)                    AS rata_rata
FROM courses c
LEFT JOIN reviews r
  ON r.course_id = c.id
  AND r.status = 'approved'
GROUP BY c.id, c.title
HAVING COUNT(r.id) > 0
ORDER BY rata_rata DESC;

```

3.4 Statistik Penjualan Bulanan

```
SELECT
    DATE_FORMAT(created_at, '%Y-%m') AS bulan,
    COUNT(*) AS total_order,
    SUM(final_price) AS total_revenue
FROM orders
WHERE status = 'completed'
GROUP BY DATE_FORMAT(created_at, '%Y-%m')
ORDER BY bulan DESC;
```

4. Subquery

4.1 Kursus dengan Harga di Atas Rata-rata

```
SELECT id, title, price
FROM courses
WHERE price > (SELECT AVG(price) FROM courses WHERE status = 'active')
    AND status = 'active'
ORDER BY price DESC;
```

4.2 User yang Belum Pernah Membeli

```
SELECT id, name, email, created_at
FROM users
WHERE role = 'user'
    AND id NOT IN (
        SELECT DISTINCT user_id FROM orders WHERE status = 'completed'
    );
```

4.3 Kursus yang Tidak Punya Review

```
SELECT c.id, c.title
FROM courses c
WHERE c.status = 'active'
    AND NOT EXISTS (
        SELECT 1 FROM reviews r WHERE r.course_id = c.id
    );
```

5. Query Dashboard Bisnis

5.1 Dashboard Admin — Statistik Utama

```

SELECT
  (SELECT COUNT(*) FROM users WHERE role = 'user')           AS
total_student,
  (SELECT COUNT(*) FROM users WHERE role = 'instructor')     AS
total_instructor,
  (SELECT COUNT(*) FROM courses WHERE status = 'active')     AS
kursus_aktif,
  (SELECT COUNT(*) FROM orders WHERE status = 'completed')   AS
total_transaksi,
  (SELECT SUM(final_price) FROM orders WHERE status = 'completed') AS
total_revenue;

```

5.2 Dashboard Instruktur — Penjualan Saya

```

-- Ganti :instructor_id dengan ID instruktur yang login
SELECT
  c.title,
  COUNT(o.id)           AS terjual,
  SUM(o.final_price)    AS revenue,
  AVG(r.rating)         AS rata_rating
FROM courses c
LEFT JOIN orders o
  ON o.course_id = c.id AND o.status = 'completed'
LEFT JOIN reviews r
  ON r.course_id = c.id AND r.status = 'approved'
WHERE c.instructor_id = :instructor_id
GROUP BY c.id, c.title
ORDER BY revenue DESC;

```

5.3 Dashboard Student — Progress Belajar

```

-- Berapa % kursus yang sudah saya selesaikan?
SELECT
  c.id,
  c.title,
  COUNT(DISTINCT cl.id)           AS total_lecture,
  COUNT(DISTINCT lc.lecture_id)   AS lecture_selesai,
  ROUND(
    COUNT(DISTINCT lc.lecture_id) * 100.0 /
    NULLIF(COUNT(DISTINCT cl.id), 0),
    1) AS progress_persen
FROM enrollments e
JOIN courses c      ON c.id = e.course_id
JOIN course_sections cs ON cs.course_id = c.id
JOIN course_lectures cl ON cl.section_id = cs.id
LEFT JOIN lecture_completions lc
  ON lc.lecture_id = cl.id
  AND lc.user_id = e.user_id

```

```
WHERE e.user_id = :user_id
GROUP BY c.id, c.title;
```

5.4 Validasi Coupon Saat Checkout

```
SELECT
    id, code, discount_percent, valid_until,
    max_usage, used_count, status
FROM coupons
WHERE code      = :input_code
    AND status  = 1
    AND valid_until >= CURDATE()
    AND (max_usage IS NULL OR used_count < max_usage)
LIMIT 1;
```

6. Query Lanjutan

6.1 Window Function — Ranking Instruktur per Kategori

```
SELECT *
FROM (
    SELECT
        cat.name          AS kategori,
        u.name             AS instruktur,
        SUM(o.final_price) AS revenue,
        RANK() OVER (
            PARTITION BY cat.id
            ORDER BY SUM(o.final_price) DESC
        ) AS ranking
    FROM users u
    JOIN courses c    ON c.instructor_id = u.id
    JOIN categories cat ON c.category_id   = cat.id
    JOIN orders o
        ON o.course_id = c.id
        AND o.status    = 'completed'
    GROUP BY cat.id, cat.name, u.id, u.name
) AS sub
WHERE ranking <= 3 -- top 3 per kategori
ORDER BY kategori, ranking;
```

6.2 CTE — Funnel Konversi

"Berapa user yang tambah ke cart vs. yang benar-benar beli?"

```
WITH user_cart AS (
    SELECT DISTINCT user_id FROM carts
```

```

),
user_buy AS (
    SELECT DISTINCT user_id FROM orders WHERE status = 'completed'
)
SELECT
    (SELECT COUNT(*) FROM user_cart) AS jumlah_pakai_cart,
    (SELECT COUNT(*) FROM user_buy) AS jumlah_beli,
    ROUND(
        (SELECT COUNT(*) FROM user_buy) * 100.0 /
        NULLIF((SELECT COUNT(*) FROM user_cart), 0),
        2) AS konversi_persen;

```

6.3 Transaksi: Checkout End-to-End

Saat user checkout, kita harus menjamin **atomicity** (semua sukses atau semua rollback):

```

START TRANSACTION;

-- 1. Buat payment
INSERT INTO payments (user_id, midtrans_order_id, total_amount, status,
created_at, updated_at)
VALUES (3, 'OD-2026-0001', 350000, 'pending', NOW(), NOW());

SET @payment_id = LAST_INSERT_ID();

-- 2. Buat orders (multi-kursus)
INSERT INTO orders (payment_id, user_id, course_id, instructor_id, original_price,
final_price, status, created_at, updated_at)
VALUES
    (@payment_id, 3, 1, 2, 200000, 200000, 'pending', NOW(), NOW()),
    (@payment_id, 3, 5, 4, 150000, 150000, 'pending', NOW(), NOW());

-- 3. Kosongkan cart user
DELETE FROM carts WHERE user_id = 3 AND course_id IN (1, 5);

COMMIT;
-- Kalau ada error di tengah → ROLLBACK; → semua dibatalkan

```

7. Query Maintenance (Bonus)

```

-- Daftar kupon kadaluarsa yang masih aktif (bug data)
SELECT * FROM coupons
WHERE status = 1 AND valid_until < CURDATE();

-- Hitung total kursus yang terdaftar tapi belum diakses sama sekali
SELECT c.title, e.user_id
FROM enrollments e
JOIN courses c ON c.id = e.course_id

```



```
WHERE NOT EXISTS (  
    SELECT 1  
    FROM course_sections cs  
    JOIN course_lectures cl ON cl.section_id = cs.id  
    JOIN lecture_completions lc  
        ON lc.lecture_id = cl.id  
        AND lc.user_id = e.user_id  
    WHERE cs.course_id = c.id  
);
```

✓ Kesimpulan Query

Skema BelajarKUY mendukung berbagai pola query yang dibutuhkan sistem nyata:

- ✓ **CRUD** untuk operasi sehari-hari
- ✓ **JOIN** untuk menggabung data antar tabel relasional
- ✓ **Agregasi** untuk laporan & dashboard
- ✓ **Subquery & CTE** untuk analisa lanjutan
- ✓ **Transaksi** untuk operasi multi-langkah (checkout)
- ✓ **Window Function** untuk ranking & analitik

Database **terbukti fleksibel** menjawab kebutuhan dari operasional sampai BI/analytics.